

Experiment 2: Compiler Hardening Flags and UBSan/ASan Diagnostics

Harsh Sen (Roll No. 2301730194)

Aim

To compile the type-conversion program from Experiment 1 under four configurations — no flags, `-Wall -Wextra`, `-fsanitize=undefined` (UBSan), and `-fsanitize=address` (ASan) — to capture the compiler output and runtime diagnostics for each, to record which warnings appeared, which runtime errors were caught, and which defects each configuration missed, and to explain how each flag contributes to catching insecure code.

Tools / Environment

GCC 13.3.0 (Ubuntu 24.04), the program under test being `vulnerable.c` from Experiment 1 (unchecked `int→short`, `int→unsigned int`, `float→int`, and `atoi()` string-to-numeric conversions).

Program Under Test

The exact same `vulnerable.c` from Experiment 1 was used, unmodified, for all four configurations, so that any difference in output is attributable only to the compiler/runtime flags and not to a change in the source code.

The Four Compilation Commands

1. No flags

```
gcc src/vulnerable.c -o v1_noflags
```

2. Warnings enabled

```
gcc -Wall -Wextra src/vulnerable.c -o v2_wall
```

3. Undefined Behaviour Sanitizer

```
gcc -g -fsanitize=undefined src/vulnerable.c -o v3_ubsan
```

4. Address Sanitizer

```
gcc -g -fsanitize=address src/vulnerable.c -o v4_asan
```

All four commands compiled with **exit code 0** and produced **no compiler errors**.

Side-by-Side Diagnostic Output

Compiler output (stderr during compilation)

Configuration	Compiler warnings emitted
No flags	<i>(none)</i>
-Wall -Wextra	<i>(none)</i>
-fsanitize=undefined	<i>(none)</i>
-fsanitize=address	<i>(none)</i>

All four configurations compiled with zero warnings. This is a result in itself and is explained in the Analysis section: the narrowing/sign-changing conversions in the source are written with *explicit* casts, which GCC treats as programmer intent and does not flag even under -Wall -Wextra (or -Wconversion, tested separately below).

Runtime output

The runtime stdout/stderr produced by configurations 1, 2, 3 and 4 was **byte-for-byte identical** in all four cases (verified with diff):

```
=====
EXPERIMENT 1 - UNSAFE / VULNERABLE DATA TYPE CONVERSIONS
=====

---- [1] int -> short (truncation) ----
Before (int)      : 70000
After (short, cast) : 4464  <-- WRONG, wrapped around

---- [2] signed int -> unsigned int (sign conversion) ----
Before (int, signed) : -1
After (unsigned int) : 4294967295  <-- WRONG, became huge positive

---- [3] float -> int (overflow) ----
Before (float) : 10000000000.00
After (int) : -2147483648  <-- WRONG / undefined behaviour
```

```
---- [4] string -> numeric (atoi, unchecked) ----  
Before (string) : "12abc"  
After  (int)    : 12  <-- silently ignored trailing 'abc'  
  
Before (string) : "hello"  
After  (int)    : 0   <-- WRONG, no error reported, looks valid  
  
Before (string) : "9999999999999999999"  
After  (int)    : 1569325055  <-- WRONG, overflowed silently
```

```
=====
```

All conversions above completed WITHOUT any error checking.
Every '<-- WRONG' line is a bug a real program could ship
with no compiler error and no runtime crash.

```
=====
```

None of the four configurations aborted, printed a sanitizer report, or altered the program's behaviour in any visible way.

Supplementary Checks (to explain *why*)

Two additional, targeted tests were run to explain the negative results above rather than simply reporting “nothing happened.”

Why -Wall -Wextra (and even -Wconversion) stayed silent

Adding -Wconversion -Wsign-conversion on top of -Wall -Wextra still produced **zero warnings** on vulnerable.c. A minimal isolated test confirms the reason:

```
short truncated = big_value;           /* IMPLICIT conversion */
unsigned int converted = neg_value;     /* IMPLICIT conversion */
```

compiled with -Wconversion -Wsign-conversion produces:

```
warning: conversion from 'int' to 'short int' may change value [-Wconversion]
warning: conversion to 'unsigned int' from 'int' may change the sign
        of the result [-Wsign-conversion]
```

but the *same* conversions written with an **explicit cast** — (short)big_value, (unsigned int)neg_value, exactly as in vulnerable.c — produce no warning at all, even with -Wconversion enabled. An explicit cast is read by the compiler as “the programmer intended this,” which silences the diagnostic.

Why UBSan caught nothing by default

-fsanitize=undefined alone did not catch the out-of-range float → int cast((int)1.0e10f), even though this specific conversion is undefined behaviour under the C standard. Enabling the specific sub-check explicitly proves the bug is detectable:

```
gcc -g -fsanitize=float-cast-overflow src/vulnerable.c -o v3b_floatcast
./v3b_floatcast
```

```
src/vulnerable.c:63:9: runtime error: 1e+10 is outside the range of
representable values of type 'int'
```

So on this GCC version, plain -fsanitize=undefined does **not** automatically include the float-cast-overflow check — it must be requested by name. The other three conversions in the program (truncation, sign conversion, atoi() misparsing) are not undefined behaviour at all under the C standard — they are well-defined but semantically wrong — so no UBSan check, however configured, can flag them.

Comparison Table

Config	Compiler warnings	Runtime errors caught	Defects missed	Why
No flags	None	None	All 4 (truncation, sign-conv., float overflow, bad atoi)	No instrumentation of any kind
-Wall -Wextra	None	None	All 4	Conversions use explicit casts, which suppress conversion warnings even under -Wconversion
UBSan (-fsanitize=undefined)	None	None by default; float-cast-overflow catchable only if enabled explicitly	3 conversions that are well-defined-but-wrong, plus the float bug unless the specific sub-check is added	UBSan targets <i>undefined behaviour</i> only, and on GCC the “undefined” group does not pull in every individual check
ASan (-fsanitize=address)	None	None	All 4	ASan targets memory-safety violations (buffer overflow, use-after-free); this program has no memory access bug for it to detect

Analysis — How Each Flag Contributes to Catching Insecure Code

No flags. Represents the default, unhardened build most students and even production systems ship with unless a build system is configured otherwise. It provides no protection whatsoever against integer conversion bugs.

-Wall -Wextra. Extremely valuable for a broad class of mistakes — unused variables, mismatched types in *implicit* conversions, suspicious comparisons — but these warnings are a **compile-time, static** check based on how the code is written, not what it does. A programmer who “fixes” a warning by adding an explicit cast has told the compiler to stop checking that line, whether or not the underlying value is actually safe to convert. This experiment demonstrates that gap concretely: the exact same bugs that -Wconversion catches when written implicitly become invisible once wrapped in a cast.

UBSan (-fsanitize=undefined). A **runtime, dynamic** check that instruments the binary to detect specific categories of undefined behaviour (signed overflow, invalid shifts, out-of-range float-to-int casts, misaligned/null pointer use, etc.) as they actually occur during execution, with an exact source line in the report. Its value is high but scoped: it only catches behaviour

the C standard defines as *undefined*, and even within that scope, this experiment shows that not every sub-check is enabled by the umbrella `-fsanitize=undefined` group on every compiler — specific checks like `float-cast-overflow` may need to be requested explicitly. It cannot catch conversions that are well-defined by the standard but simply wrong for the program's logic.

ASan (`-fsanitize=address`). A **runtime, dynamic** check aimed at an entirely different bug class: memory-safety violations such as heap/stack/global buffer overflows, use-after-free, and use-after-return. It is one of the most effective tools available for catching exploitable memory corruption, but it is not designed to say anything about pure arithmetic or type-conversion logic errors that never touch memory out of bounds. This experiment's null result for ASan is expected and correctly diagnostic of the tool's actual scope, not a failure of the tool.

Taken together, this experiment shows that no single flag or sanitizer is sufficient on its own: compiler warnings catch *style mistakes visible in the syntax* (and only when no cast hides them), UBSan catches *standard-defined undefined behaviour* (and only for the specific checks enabled), and ASan catches *memory-safety violations*. None of them is a substitute for the explicit range-checking approach used in the “safe” version from Experiment 1, which validates the actual *value* before every conversion regardless of how the conversion is written or which sanitizer is attached.

Result

All four build configurations of the unmodified Experiment-1 `vulnerable.c` compiled cleanly with zero warnings and ran to completion with byte-for-byte identical runtime output, silently producing four incorrect conversion results in every case. Enabling `-Wall -Wextra` added no protection because every unsafe conversion in the source is hidden behind an explicit cast, which was verified by reproducing the same conversions without a cast and observing that `-Wconversion/-Wsign-conversion` then do fire. Enabling `-fsanitize=undefined` also produced no diagnostic by default, because GCC's undefined umbrella group did not include the `float-cast-overflow` check needed to catch the one genuinely undefined conversion in the program (confirmed by enabling that check by name), and the remaining three conversions are well-defined behaviour that no UBSan check can flag. Enabling `-fsanitize=address` also produced no diagnostic, correctly reflecting that the program contains no memory-safety violation for ASan to detect. The experiment confirms that compiler warnings, UBSan, and ASan each address a distinct, narrow class of defect, and that explicit, value-level range checking (as implemented in the “safe” version of Experiment 1) is the only configuration tested that actually prevents — rather than merely attempts to detect — this class of data-type conversion bug.

Both the program under test and all annotated diagnostic output files for the four configurations have been pushed to the GitHub repository, tracked across two commits.